

Banco de Preguntas y Respuestas Técnicas para la Defensa

Proyecto SGDM ASCEND (Sistema de Gestión Deportiva y Multideportiva)

Bloque 1: Enrutamiento, Parámetros URL y Vistas Dinámicas

1. ¿Por qué utilizaron parámetros `id` y `section` en la URL?

Respuesta:

Utilizamos parámetros de consulta (*Query Parameters* como `?id=3§ion=fixture`) para desacoplar la interfaz del contenido y habilitar una arquitectura de **Página Dinámica Reutilizable**:

1. **Identificador del Recurso (`id`)**: Le indica al motor de JavaScript qué torneo específico debe buscar en la base de datos o almacenamiento local para renderizar su información sin necesidad de crear archivos HTML estáticos para cada torneo.
 2. **Navegación de Estado (`section`)**: Permite un **enlace profundo (*Deep Linking*)**. Un usuario puede compartir la URL directa de los resultados o del fixture de un torneo (ej. `detalle-torneo.html?id=2§ion=clasificacion`), y la aplicación cargará directamente esa pestaña activa.
 3. **Historial del Navegador**: Permite que los botones "Atrás" y "Adelante" del navegador funcionen de forma natural sin recargar toda la página física.
-

2. ¿Qué ocurre si el `id` recibido no existe?

Respuesta:

La aplicación implementa un patrón de **Manejo de Errores Defensivo (*Graceful Degradation*)**:

1. La función `obtenerTorneoPorId(id)` busca el registro. Si no encuentra coincidencia, retorna `undefined` o `null`.
 2. El controlador evalúa esta condición (`if (!torneo)`) y, en lugar de romper la ejecución con errores en consola (*Cannot read properties of undefined*), ejecuta una rutina de contingencia:
 - Muestra un mensaje amigable al usuario indicando: "El torneo solicitado no existe o ha sido dado de baja".
 - Opcionalmente redirige al catálogo general (`torneos.html`) o renderiza un componente de estado vacío (*Empty State*).
 3. Esto cumple con la norma de usabilidad y robustez exigida por estándares de calidad de software (ISO/IEC 25010).
-

3. ¿Por qué el `id` es numérico y no el nombre del torneo?

Respuesta:

Se optó por identificadores numéricos enteros autoincrementales por motivos de **integridad de datos, rendimiento y normalización relacional**:

1. **Inmutabilidad:** El nombre de un torneo puede cambiar (ej. corregir una falta de ortografía o cambiar el patrocinador: "Copa Verano" → "Copa Verano Antel"). Si la URL o las relaciones dependieran del nombre, cualquier cambio rompería los enlaces y las claves foráneas (*Foreign Keys*).
 2. **Rendimiento en Base de Datos:** Los índices sobre enteros (**INT PRIMARY KEY**) ocupan solo 4 bytes, se comparan en un solo ciclo de CPU y son infinitamente más veloces en operaciones **JOIN** y **WHERE** que comparar cadenas de texto largas (**VARCHAR**).
 3. **Manejo de Caracteres Especiales:** Los nombres suelen contener espacios, tildes y símbolos que requieren codificación URL (*URL Encoding* como **%20**, **%C3%A1**), lo que ensucia los enlaces y complica el parseo.
-

4. ¿Qué ventajas tiene reutilizar una sola página HTML para todos los torneos?

Respuesta:

Esta técnica representa el principio fundamental **DRY (Don't Repeat Yourself - No te repitas)**:

1. **Mantenibilidad Extrema:** Si mañana queremos rediseñar la cabecera o agregar una nueva pestaña al detalle de torneos, solo modificamos un único archivo (**detalle-torneo.html**). Si tuviéramos 50 archivos HTML para 50 torneos, tendríamos que editar los 50 manualmente.
 2. **Escalabilidad Infinita:** La plataforma puede tener 1, 100 o 10.000 torneos sin crear un solo archivo HTML adicional; el HTML actúa como una **plantilla estandarizada (template)** que se puebla dinámicamente con datos.
 3. **Ahorro de Ancho de Banda y Caché:** El navegador descarga y cachea la plantilla y sus hojas de estilo una sola vez, descargando únicamente los datos livianos en formato JSON.
-

Bloque 2: Arquitectura Modular, JS y CSS

5. ¿Por qué modularizaron el proyecto en varios archivos JS?

Respuesta:

Se aplicó el principio de **Separación de Responsabilidades (Separation of Concerns)**:

1. **Especialización:** Cada archivo tiene una única tarea clara:
 - **almacenamiento.js:** Capa de persistencia y acceso a datos (*Data Access Layer*).
 - **render.js:** Construcción y manipulación de elementos en el DOM (*View Layer*).
 - **controlador.js** o **detalle-torneo.js:** Lógica de negocio, captura de eventos y flujo.
 2. **Trabajo en Equipo sin Conflictos:** En un equipo de varios desarrolladores, la modularización permite que un programador trabaje en la interfaz mientras otro mejora las funciones de cálculo sin pisarse en Git (*Merge Conflicts*).
 3. **Reutilización y Pruebas:** Funciones utilitarias como formatear fechas o validar correos pueden ser importadas y reutilizadas en múltiples vistas.
-

6. ¿Por qué dividieron el CSS en varios archivos?

Respuesta:

Para implementar una arquitectura de estilos inspirada en metodologías modernas (**SMACSS** e **ITCSS**):

1. **variables.css:** Define los *Design Tokens* universales (paleta de colores, tipografías, radios de borde).

2. **base.css / reset.css**: Normalización de estilos de fábrica del navegador y tipografías globales.
 3. **layout.css**: Estructuras mayores de la página (header, sidebar, footer, grillas de contenedores).
 4. **componentes.css**: Estilos de piezas de interfaz aisladas y reutilizables (tarjetas, botones, modales, tablas).
 5. **CSS específicos de módulo** (ej. `admin/`, `organizador/`): Reglas exclusivas de cada panel para evitar colisiones de nombres (*CSS Specificity Wars*) y mantener el código ordenado y escalable.
-

7. ¿Por qué reutilizan `navbar` y `footer`?

Respuesta:

Por tres razones fundamentales:

1. **Consistencia en la Experiencia de Usuario (UX)**: El usuario percibe una interfaz profesional y predecible donde los puntos de navegación e identidad de marca no cambian de posición entre pantallas.
 2. **Centralización del Mantenimiento**: Si se agrega un nuevo enlace institucional o se actualiza el copyright en el footer, el cambio se refleja instantáneamente en toda la aplicación.
 3. **Preparación para Backend**: En la transición a PHP, estos componentes se convierten en plantillas maestras (`cabecera.php`, `pie_pagina.php`) incluidas con `require_once`, consolidando la arquitectura modular.
-

Bloque 3: Persistencia Local y Almacenamiento

8. ¿Qué función cumple `localStorage` en ASCEND?

Respuesta:

En la fase de Frontend (previa al backend en servidor), `localStorage` actúa como un **Mock de Base de Datos del Lado del Cliente (*Client-Side Mock Database*)**:

1. Permite la persistencia de datos entre recargas de página y cambios de pestaña (creación de nuevos torneos, registro de usuarios, equipos y carga de marcadores).
 2. Permite simular el ciclo de vida completo de un sistema CRUD (*Create, Read, Update, Delete*) de forma interactiva y funcional en el navegador.
 3. Almacena preferencias de usuario en formato de texto serializado JSON.
-

9. ¿Qué limitaciones tiene `localStorage`?

Respuesta:

`localStorage` es una solución exclusiva para prototipado o preferencias menores debido a sus limitaciones críticas:

1. **Capacidad Reducida**: Tiene un límite estricto de aproximadamente **5 MB** por dominio.
2. **Inseguridad Total (Vulnerable a XSS)**: Cualquier script malicioso inyectado puede leer el `localStorage` con `localStorage.getItem()`. Por tanto, **nunca deben almacenarse contraseñas, tokens JWT de sesión ni datos bancarios**.
3. **Naturaleza Síncrona y Bloqueante**: Las operaciones de lectura/escritura bloquean el hilo principal de JavaScript (*Main Thread*), lo que causaría congelamientos de interfaz si los datos fueran masivos.

4. **Almacena Solo Cadenas de Texto (DOMString)**: Obliga a realizar conversiones constantes con `JSON.stringify()` y `JSON.parse()`.
5. **No es Compartido entre Dispositivos**: Los datos guardados en una computadora no existen en el celular del usuario (carece de servidor centralizado).

Bloque 4: Manipulación del DOM y Eventos en JavaScript

10. ¿Qué diferencia hay entre `querySelector()`, `getElementById()` y `querySelectorAll()`?

Respuesta:

Método	Qué Retorna	Tipo de Búsqueda	Rendimiento
<code>getElementById('id')</code>	Un solo elemento HTML (Element) o null .	Busca estrictamente por el atributo id . No lleva # .	Es el más rápido de todos (acceso directo en el árbol DOM).
<code>querySelector('selector')</code>	El primer elemento que coincida con el selector CSS o null .	Acepta cualquier selector CSS complejo (.clase , #id , div > p.activo).	Muy versátil y flexible.
<code>querySelectorAll('selector')</code>	Una lista estática de nodos (NodeList) con todas las coincidencias.	Acepta cualquier selector CSS múltiple.	Permite iterar con .forEach() sobre colecciones de elementos.

11. ¿Cuándo usarías `querySelectorAll()`?

Respuesta:

Se utiliza cuando necesitamos seleccionar y manipular un **conjunto múltiple de elementos homogéneos** en el DOM.

Casos típicos en ASCEND:

1. **Pestañas de Navegación**: Para seleccionar todos los botones de las pestañas (**.btn-tab**) y asignarles a todos un **addEventListener** en bucle.
2. **Formularios Dinámicos**: Para seleccionar todos los inputs con la clase **.campo-obligatorio** y validar que ninguno esté vacío.
3. **Filtrado de Tarjetas**: Para recorrer todas las tarjetas de torneos (**.tarjeta-torneo**) y ocultar o mostrar las que coincidan con la categoría elegida por el usuario.

12. ¿Qué hace `addEventListener()`?

Respuesta:

Es el método estándar de la especificación W3C para registrar un **Escuchador de Eventos (Event Listener)** sobre un objeto del DOM:

- **Firma:** `elemento.addEventListener(tipoEvento, funcionCallback, opciones);`
 - **Mecanismo:** Vincula una función para que se ejecute de forma asíncrona cuando ocurre una acción específica del usuario o del sistema (como `'click'`, `'submit'`, `'change'`, `'input'`, `'keydown'`).
 - **Ventaja sobre atributos HTML (`onclick=""`):** Permite asociar múltiples escuchadores al mismo evento sin sobrescribirse, mantiene el código JavaScript completamente separado del marcado HTML y permite capturar el objeto `event` (para hacer `e.preventDefault()`, por ejemplo).
-

13. ¿Por qué usar `innerHTML` y no `createElement()` en determinadas vistas?

Respuesta:

Es una decisión de **equilibrio entre legibilidad, productividad y rendimiento**:

- **`innerHTML` (con Template Literals ` `):**
 - **Ventajas:** Permite maquetar fragmentos complejos de HTML anidado (tarjetas con imágenes, badges, listas y botones) de forma muy visual y rápida usando interpolación de variables (`${torneo.nombre}`).
 - **Cuándo se usa:** Para renderizar catálogos grandes, tablas y plantillas donde la estructura visual es extensa.
 - **`createElement()` + `appendChild()`:**
 - **Ventajas:** Más seguro contra ataques XSS (*Cross-Site Scripting*), no destruye ni reparsa todo el árbol interno de nodos, y conserva los eventos ya vinculados a elementos existentes.
 - **Cuándo se usa:** Para agregar nodos individuales o dinámicos donde se requiere vincular listeners directamente al elemento creado.
-

Bloque 5: Lógica de Datos, Métodos de Array y Tipos

14. ¿Qué hace `URLSearchParams`?

Respuesta:

Es una interfaz nativa moderna de JavaScript que permite parsear y manipular fácilmente los parámetros de consulta (*Query String*) de una URL:

```
// Si la URL es: detalle-torneo.html?id=3&section=equipos
const parametros = new URLSearchParams(window.location.search);
const idTorneo = parametros.get('id');           // Retorna "3" (como string)
const seccion = parametros.get('section');       // Retorna "equipos"
```

Evita tener que escribir expresiones regulares o dividir manualmente la cadena con `.split('?')` y `.split('&')`.

15. ¿Por qué convierten el `id` con `Number()`?

Respuesta:

Porque los valores extraídos de la URL mediante `URLSearchParams.get('id')` o de inputs HTML son **siempre cadenas de texto** (`string`) (ej. `"3"`).

- Si comparamos con igualdad estricta: `"3" === 3` el resultado es `false`.
- Al convertir con `Number(id)` o `parseInt(id, 10)`, garantizamos que el tipo de dato sea un número primitivo, permitiendo comparaciones seguras (`item.id === idNumerico`), operaciones aritméticas e indexación sin errores sutiles de tipo (*Type Coercion*).

16. ¿Qué hace `obtenerTorneoPorId()`?

Respuesta:

Es una función de la **capa de acceso a datos** que encapsula la búsqueda de un objeto torneo específico dentro de la colección:

```
function obtenerTorneoPorId(id) {
  const listaTorneos = obtenerTorneosLocalStorage();
  return listaTorneos.find(t => t.id === Number(id));
}
```

Abstrae la lógica de almacenamiento: si mañana cambiamos `localStorage` por una llamada `fetch()` a una API PHP, los componentes que invocan a `obtenerTorneoPorId()` no sufren modificaciones en su estructura.

17. ¿Por qué usan `find()` y no `filter()`?

Respuesta:

Porque el `id` de un torneo es una **clave primaria única (Unique Identifier)**:

1. **Semántica y Retorno:** `find()` devuelve el **único objeto directamente** (o `undefined`), mientras que `filter()` devuelve un **Array de elementos**. Con `filter()` tendríamos que escribir `resultado[0]`.
2. **Eficiencia en Tiempo de Ejecución (Early Exit):** `find()` se detiene inmediatamente al encontrar la primera coincidencia (complejidad $O(k)$). `filter()` está obligado a recorrer todo el array hasta el último elemento aunque lo haya encontrado en la primera posición ($O(n)$).

18. ¿Qué diferencia hay entre `find()` y `filter()`?

Respuesta:

Característica	<code>Array.prototype.find()</code>	<code>Array.prototype.filter()</code>
Valor de Retorno	El primer elemento que cumpla la condición, o <code>undefined</code> .	Un nuevo Array con todos los elementos que cumplan la condición (puede ser <code>[]</code> vacío).
Recorrido	Se corta en el momento que encuentra el primer <code>true</code> (<i>Cortocircuito</i>).	Recorre el 100% del array obligatoriamente.
Caso de Uso	Buscar por ID, buscar usuario por email único.	Filtrar torneos por disciplina, listar partidos de una ronda.

Bloque 6: Ciclo de Vida, Referencias y Variables en JS

19. ¿Qué ocurre si un script se ejecuta antes de cargar el DOM?

Respuesta:

Si el script se ejecuta antes de que el navegador termine de parsear el árbol HTML:

1. Cualquier llamada a `document.getElementById()` o `querySelector()` sobre elementos que están más abajo en el archivo HTML retornará `null`.
 2. Al intentar acceder a propiedades de ese elemento (ej. `btn.addEventListener()`), JavaScript lanzará una excepción crítica:
`Uncaught TypeError: Cannot read properties of null (reading 'addEventListener')` deteniendo la ejecución del resto del script.
 3. **Soluciones implementadas:** Colocar las etiquetas `<script>` al final del `<body>`, usar el atributo `defer` en el `<head>`, o envolver la inicialización dentro del evento `document.addEventListener('DOMContentLoaded', ...)`.
-

20. ¿Por qué guardan referencias al DOM en un objeto?

Respuesta:

Por un patrón de optimización llamado **DOM Caching (Caché de Elementos del DOM)**:

```
const DOM = {  
  contenedor: document.getElementById('contenedor-torneo'),  
  titulo: document.getElementById('titulo-torneo'),  
  btnGuardar: document.getElementById('btn-guardar')  
};
```

1. **Rendimiento:** Consultar el DOM (`document.querySelector`) es una de las operaciones más lentas en el navegador porque atraviesa el árbol C++ del motor de render. Guardar la referencia en memoria una sola vez evita búsquedas repetidas.
 2. **Legibilidad y Mantenimiento:** Centraliza todos los selectores en un solo lugar al inicio del archivo; si un `id` en el HTML cambia, solo se actualiza una línea en el objeto `DOM`.
-

21. ¿Qué significa que los objetos se pasan por referencia?

Respuesta:

En JavaScript, los tipos primitivos (`number`, `string`, `boolean`) se pasan **por valor** (se crea una copia independiente). En cambio, los tipos complejos (**Objetos y Arrays**) se pasan **por referencia**:

- Las variables no guardan el objeto completo, sino un **puntero a la dirección de memoria** donde reside el objeto.
 - Si pasamos un objeto torneo a una función y modificamos una de sus propiedades dentro de ella (ej. `torneo.estado = 'en_curso'`), el cambio impacta en el objeto original fuera de la función porque ambas variables apuntan a la misma posición de memoria.
-

22. ¿Qué diferencia hay entre `let` y `const`?

Respuesta:

Característica	<code>const</code>	<code>let</code>
Reasignación	No se puede reasignar. Si intentas <code>const x = 5; x = 6;</code> da error <code>TypeError</code> .	Permite reasignación (<code>let x = 5; x = 6;</code>).
Inicialización	Obligatoria al momento de declararla.	Opcional (puede declararse vacía <code>let x;</code>).
Ámbito (Scope)	Ámbito de bloque <code>{ ... }</code> .	Ámbito de bloque <code>{ ... }</code> .
Buena Práctica	Usar <code>const</code> por defecto para el 90% de variables (evita mutaciones accidentales).	Usar <code>let</code> únicamente cuando la variable deba cambiar (contadores, acumuladores).

23. ¿Se pueden modificar las propiedades de un objeto declarado con `const`?

Respuesta:

Sí, absolutamente.

`const` protege la **referencia (la asignación de la variable)**, no el contenido interno del objeto (*mutabilidad*):

```
const torneo = { id: 1, nombre: "Copa ASCEND" };

// ☒ PERMITIDO: Modificar o agregar propiedades internas
torneo.nombre = "Copa ASCEND Pro";
torneo.estado = "en_curso";

// ☐ PROHIBIDO: Reasignar la referencia a un nuevo objeto
torneo = { id: 2 }; // Lanza: TypeError: Assignment to constant variable.
```

(Para hacer un objeto completamente inmutable se requiere `Object.freeze()`).

Bloque 7: Estrategia Mobile First y Responsive Web Design

24. ¿Qué significa Mobile First?

Respuesta:

Es una filosofía de diseño y desarrollo web donde **se diseña y programa primero para la pantalla más pequeña y con mayores restricciones (teléfonos móviles)**, y luego se va expandiendo progresivamente la experiencia hacia tablets y monitores de escritorio mediante **Mejora Progresiva (Progressive Enhancement)**:

1. El CSS base (fuera de cualquier media query) contiene las reglas para pantallas móviles.
2. Obliga a priorizar el contenido esencial eliminando sobrecarga visual innecesaria.
3. Mejora los tiempos de carga en dispositivos móviles con conexiones lentas.

25. ¿Por qué utilizaron `min-width`?

Respuesta:

`min-width` es la piedra angular técnica de la estrategia **Mobile First**:

- Una regla `@media (min-width: 768px)` significa: "Aplica estos estilos desde los 768px de ancho hacia arriba (tablets y escritorios)".
 - Al usar `min-width`, los celulares leen el CSS base sin procesar media queries complejas. A medida que la pantalla crece, se van agregando columnas y detalles adicionales en cascada limpia, evitando sobrescribir estilos una y otra vez (como ocurre cuando se usa `max-width` en Desktop-First).
-

26. ¿Qué ocurre si eliminan todas las media queries?

Respuesta:

Si eliminamos todas las media queries:

1. La aplicación se visualizará en cualquier pantalla (incluyendo monitores 4K) **exactamente con el diseño de la versión móvil**.
 2. Los elementos se mantendrán apilados verticalmente en una sola columna (`flex-direction: column`), los menús laterales permanecerán colapsados en formato hamburguesa y los anchos ocuparán el 100%.
 3. **El sitio seguirá siendo 100% usable y legible** (nunca se romperá con desbordes horizontales), demostrando la solidez y resiliencia de la arquitectura Mobile First.
-

Bloque 8: Maquetación Avanzada con Flexbox y CSS Grid

27. ¿Por qué utilizan CSS Grid y cuándo Flexbox?

Respuesta:

Aplicamos la regla arquitectónica de **la herramienta adecuada para el problema adecuado**:

- **CSS Grid (Bidimensional - 2D)**: Se utiliza para matrices y estructuras que controlan simultáneamente **filas y columnas**.
 - *Casos de uso*: Catálogos de tarjetas de torneos, galerías de disciplinas, cuadrículas de llaves de eliminación (*Brackets*) y tableros estadísticos.
 - **Flexbox (Unidimensional - 1D)**: Se utiliza para componentes y flujos en un **único eje a la vez** (fila o columna).
 - *Casos de uso*: Barra superior (*Navbar*), pie de página, botones con íconos centrados, pestañas en línea y layout de sidebar + contenido elástico del Admin.
-

28. ¿Qué significa `minmax(0, 1fr)`?

Respuesta:

Es una técnica avanzada de CSS Grid para **prevenir el desborde indeseado de contenido**:

- Por defecto, una columna en Grid con `1fr` tiene un valor implícito de `min-width: auto`. Si dentro de la columna hay una tabla ancha, una imagen sin tamaño o un texto largo sin espacios, la columna se ensancha forzosamente y rompe la grilla.

- Al declarar `minmax(0, 1fr)`, le indicamos al navegador que el **tamaño mínimo permitido de la columna es 0px** y el **máximo es 1 fracción del espacio disponible**. Esto permite que los elementos hijos con `overflow: hidden` o `text-overflow: ellipsis` se ajusten perfectamente sin empujar la cuadrícula.
-

29. ¿Qué función cumple `max-width`?

Respuesta:

Establece un **techo o límite superior al crecimiento de un contenedor** garantizando la fluidez responsiva:

- A diferencia de `width: 1200px` (que es rígido y rompe pantallas móviles), `max-width: 1200px` permite que en celulares el contenedor mida el 100% del ancho del dispositivo y en monitores de alta resolución (Full HD, 2K, 4K) se detenga en 1200px.
 - Combinado con `margin: 0 auto;`, centra automáticamente el contenedor en la pantalla, respetando los estándares de ergonomía visual y legibilidad de la norma ISO 9241 (evitando que el usuario deba girar el cuello para leer de un extremo a otro).
-

30. ¿Por qué utilizar `gap` en lugar de márgenes?

Respuesta:

`gap` es la propiedad moderna de CSS para gestionar el espaciado en Flexbox y Grid:

1. **Espaciado Exclusivamente Intermedio:** Aplica separación únicamente **entre** los elementos hijos, sin agregar márgenes sobrantes en el primer ni en el último elemento.
 2. **Elimina Hacks CSS:** Evita tener que escribir selectores de parche como `:last-child { margin-right: 0; }` o márgenes negativos en el contenedor padre.
 3. **Adaptabilidad Direccional:** Si el contenedor cambia de fila a columna (`flex-direction: column`), el `gap` pasa a separar verticalmente de forma automática.
-

Bloque 9: Variables CSS, Semántica y Estructura

31. ¿Qué ventajas tienen las variables CSS (*Custom Properties*)?

Respuesta:

1. **Centralización del Sistema de Diseño (*Design Tokens*):** En `variables.css` se declaran los colores de marca, tipografías y sombras. Cambiar un color allí actualiza instantáneamente cientos de componentes.
 2. **Soporte Dinámico para Temas:** Permite implementar Modo Oscuro (*Dark Mode*) o temas personalizados cambiando simplemente una clase en el `<body>` o manipulando las variables con JavaScript en tiempo real.
 3. **Legibilidad del Código:** Es mucho más semántico y autodocumentado leer `color: var(--color-primario);` que descifrar un código hexadecimal crudo como `#F511CB`.
-

32. ¿Por qué utilizar etiquetas semánticas HTML5?

Respuesta:

El uso de etiquetas como `<header>`, `<nav>`, `<main>`, `<section>`, `<article>`, `<aside>` y `<footer>` en lugar de `<div>` genéricos aporta tres pilares técnicos:

1. **Accesibilidad Universal (A11y / W3C):** Los lectores de pantalla para personas con discapacidad visual pueden saltar directamente a la navegación o al contenido principal.
 2. **Optimización para Motores de Búsqueda (SEO):** Los motores de indexación de Google y Bing comprenden la jerarquía y relevancia del contenido de la plataforma.
 3. **Mantenibilidad y Estructura Limpia:** Facilita la lectura del código fuente por parte de otros programadores del equipo.
-

33. ¿Qué ventajas aporta la organización por carpetas?

Respuesta:

La estructura de carpetas implementada (`codigo_fuente/` dividida en `configuracion/`, `modelos/`, `controladores/`, `vistas/`, `ayudantes/` y `publico/`) aporta:

1. **Adherencia al Patrón Arquitectónico MVC:** Cada carpeta aloja exclusivamente archivos de una capa específica, garantizando orden formal.
 2. **Seguridad en Servidor:** El archivo de entrada `publico/index.php` es el único accesible públicamente; los modelos y configuraciones con credenciales de base de datos quedan protegidos fuera de la raíz web pública.
 3. **Localización Inmediata:** Cualquier desarrollador nuevo sabe exactamente dónde encontrar un controlador, un ayudante de validación o un estilo CSS.
-

34. ¿Cómo favorece la arquitectura al trabajo en equipo?

Respuesta:

Favorece la productividad colaborativa mediante:

1. **Desacoplamiento Modular:** Cada uno de los 4 integrantes del equipo puede asumir un módulo independiente (Administrador, Organizador, Jugador/Equipos, Torneos/Fixtures) sin pisar los archivos de los demás.
 2. **Contratos e Interfaces Claras:** La base de datos normalizada en 3FN y las funciones helper estandarizadas (`Sesion::obtenerUsuarioId()`, `Validador::campoRequerido()`) permiten que las funciones de un módulo se comuniquen fluidamente con los otros.
 3. **Reducción de Conflictos en Git:** Al no existir archivos monolíticos gigantes, los *pull requests* y *merges* se integran limpiamente.
-

Bloque 10: Decisiones Arquitectónicas, Rendimiento y Rúbrica

35. ¿Qué decisión técnica consideran la más importante del proyecto?

Respuesta:

La decisión más trascendental fue **la normalización en 3ra Forma Normal (3FN) con Herencia de Tablas (Class Table Inheritance 1:1) para los Usuarios:**

- Dividir la identidad en una tabla padre (**usuarios**) y dos tablas hijas especializadas (**perfiles_jugadores** y **perfiles_organizadores**) resolvió de forma elegante el problema de los roles sin duplicar código ni permitir columnas fantasma con valores **NULL**.
 - Esta base de datos sólida garantizó que tanto el Frontend como el Backend en PHP se construyeran sobre cimientos limpios, seguros y escalables.
-

36. ¿Cómo mejorarían el rendimiento con miles de torneos?

Respuesta:

Implementaríamos tres estrategias técnicas probadas:

1. **Paginación y Carga Perezosa (*Lazy Loading / Infinite Scroll*)**: En lugar de consultar miles de torneos de golpe, traer lotes de 12 torneos mediante **LIMIT 12 OFFSET 0** en SQL.
 2. **Indexación Estratégica en Base de Datos**: Creación de índices compuestos (**CREATE INDEX idx_torneos_busqueda ON torneos(estado, juego_id, fecha_inicio)**), permitiendo búsquedas en milisegundos.
 3. **Caché en Servidor y CDN**: Cachear en memoria (Redis o Memcached) los torneos más consultados y servir imágenes y banners estáticos a través de una red de distribución de contenidos (CDN).
-

37. ¿Por qué los roles están separados?

Respuesta:

Por **Seguridad, Principio de Menor Privilegio (PoLP) y Control de Acceso Basado en Roles (RBAC)**:

1. **Aislamiento de Privilegios**: Un Jugador no debe tener acceso a las rutas ni a las vistas de aprobación de torneos o gestión de usuarios que corresponden al Administrador.
 2. **Especialización de Datos**: Un jugador tiene gamertag, estadísticas de juego y nivel; un organizador tiene razón social, localidad y verificación oficial. Separar los roles y perfiles evita mezclar lógicas dispares y previene vulnerabilidades de elevación de privilegios (OWASP Top 10: *Broken Access Control*).
-

38. ¿Qué problemas encontraron al adaptar el proyecto a Mobile First?

Respuesta:

Los dos desafíos principales fueron:

1. **Tablas de Datos Densas**: Las tablas con muchas columnas (como las llaves de eliminación o los registros de auditoría) desbordaban en pantallas de 360px.
 2. **Navegación y Menús Complejos**: La barra lateral del Administrador con múltiples submenús ocupaba toda la pantalla móvil si no se diseñaba un sistema de colapso eficiente.
-

39. ¿Cómo solucionaron los problemas responsive?

Respuesta:

1. **Contenedores de Desplazamiento Controlado**: Envolviendo las tablas en contenedores con **overflow-x: auto**; y transformando las tarjetas de torneos en bloques apilados verticales mediante Flexbox.

2. **Menú Hamburguesa Accesible:** Implementación de una barra móvil compacta (`.barra-movil`) con menú desplegable activado por JavaScript que no interfiere con el contenido central.
 3. **Unidades Relativas y Fluidas:** Reemplazo de anchos fijos en píxeles por porcentajes, fracciones `fr`, `rem` y `clamp()` para tipografías dinámicas.
-

40. ¿Qué aspectos de la rúbrica consideran mejor cumplidos por el proyecto?

Respuesta:

1. **Arquitectura y Modularidad:** Cumplimiento riguroso del patrón MVC, separación nítida entre lógica, datos y presentación, y hojas de estilo desacopladas.
2. **Calidad y Normalización de Base de Datos:** Modelo relacional estricto en 3ra Forma Normal (3FN), integridad referencial con cascadas precisas y herencia de roles.
3. **Responsive Design y Mobile First:** Adaptabilidad total comprobada desde pantallas móviles de 320px hasta monitores 4K sin desbordes.
4. **Seguridad y Buenas Prácticas:** Estructura preparada contra inyecciones SQL mediante PDO Prepared Statements, validaciones de entrada en doble capa (Front/Back) y registros inmutables de auditoría (OWASP).